

Kick Start 2020 - Round F

Analysis: ATM Queue

Firstly, denote K_i as the number of times a person will use the ATM. Formally, $K_i = \lceil A_i / X \rceil$.

Test Set 1

We can directly simulate the process using a queue.

Assume that i -th person, that wants to withdraw A_i , is first in the queue. There are two possibilities:

- $A_i \leq X$. In that case, this person withdraws A_i and leaves the queue. We can add i to the answer.
- $A_i > X$. In that case, this person withdraws X (thus setting A_i to $A_i - X$) and goes back to the end of the queue.

Time complexity of this simulation is $O(\sum K_i)$.

In the worst case, when $X = 1$, $K_i = A_i$. Since $A_i \leq 100$, the worst time complexity is $O(N \times 100)$, which easily fits into the time limit.

Test Set 2

In the second test set, K_i can be as big as 10^9 , so direct simulation is too slow.

Let's look at two people i and j . When will i -th person leave the queue before j -th person? There are two cases:

- $K_i < K_j$. Since i -th person will use the ATM fewer times than j -th person, they will leave the queue earlier.
- $K_i = K_j$ and $i < j$. If they both use the ATM the same amount of times, the person earlier in the queue in the initial configuration will leave first.

This observation is enough to form a full solution. Sort people first in ascending order of K_i , and in case of ties in ascending order of their number. After sorting, this is our answer.

Time complexity of this solution is $O(N \log N)$.

Analysis: Metal Harvest

Consider given time intervals as an array. We will refer to the start time of i -th interval as S_i and to the end time of i -th interval as E_i .

Let us sort the time interval array by start time in ascending order. To start the harvesting, we need to deploy our first robot at the start time of the first interval (S_1) in the sorted array. It is not optimal to deploy the robot before S_1 because we would be wasting robot hours. Per the problem statement, our first robot can be deployed from S_1 to $S_1 + K$ before it returns for calibration. Let us define interval length of i -th interval as $\text{interval_length}_i = E_i - S_i$.

The following cases are possible for our first robot:

1. $\text{interval_length}_1 > K$: In this case, as soon as our first robot finishes, we need to deploy another robot to cover the remaining part of the interval.
2. $\text{interval_length}_1 \leq K$: This can be further divided in two cases:
 - a. Bring the first robot back as soon as the first interval finishes. We cannot re-deploy this robot since a robot can only harvest for consecutive units of time.
 - b. Allow the first robot to be available until K units of time has elapsed. This is a better strategy since it allows us to utilize the robot for future harvesting.

Based on the above scenarios, the optimal strategy is to deploy a robot at the beginning of the first harvesting interval, for K hours. If after K hours the robot is inside the first or any subsequent harvesting interval, deploy another robot to replace it. Otherwise, deploy another robot as soon as the next harvesting interval begins. Repeat this for every interval and keep count of the number of total deployments.

Test Set 1

For this test set we can loop over sorted time intervals and then have another inner loop to go over each point in time of such interval.

At the i -th point in time, if there is no robot currently deployed for harvesting, we can deploy a robot and keep it harvesting for K units of time. If a robot is already currently deployed for harvesting then no action is needed. The total number of robots deployed is the required answer.

Since we are sorting the intervals array and then iterating over each point in time inside an interval, the worst case time complexity will be $O(\sum_{1 \leq i \leq N} \text{interval_length}_i + N \log N)$, where N is the total number of intervals and $\text{interval_length}_i \leq 200$.

Test Set 2

To solve for this test case we need to find a way to count the number of robots needed for an interval without iterating over each point in time.

Let us say we deploy a robot at S_1 . The number of robots needed for the first interval can be calculated as $\lceil \text{interval_length}_1 / K \rceil$.

Let us define a variable `last_harvest_time` as the last point in time covered by all deployed robots and initialise it as 0. It is possible that the last robot deployed to cover the first interval will go beyond E_1 hence the `last_harvest_time` after covering the first interval will be,
 $\text{last_harvest_time} = \max(\text{last_harvest_time}, S_1) + \text{number of robots needed for the first interval} * K$.

For subsequent intervals:

1. If the interval is already fully covered with the last_harvest_time, no action is needed.
2. If the interval is not fully covered by the last_harvest_time then follow a similar strategy as first interval to calculate the number of new robots required and again update the last_harvest_time. Also increment the answer by total number of new robots.

Since we are sorting the intervals array and then iterating over all intervals at once, the worst case time complexity will be $O(N \log N)$, where **N** is the total number of intervals.

Analysis: Painters' Duel

Test Set 1

There are only 4 rooms, (1, 1), (2, 1), (2, 2) and (2, 3). Both the players, will try to paint the room (2, 2), painting which would block the other player to paint more rooms.

Thus there are five cases,

- If $C = 2$, none of the players can move, therefore the answer is 0.
- If any of the players starts on room (2, 2) and $C \neq 2$, they will paint two rooms and the other player will paint one room.
- If room (2, 2) is blocked, none of the players can move, therefore the answer is 0.
- If any room other than (2, 2) is blocked, and no player start on (2, 2), then Alma will paint two rooms and Berthe will paint one room.
- If no room is blocked, and no player start on (2, 2), then Alma will paint three rooms and Berthe will paint one room.

Test Set 2

At any point during the game, few rooms of the museum might be under construction, few might have been painted by Alma, few might have been painted by Berthe and a few might be unpainted.

The rooms which are either under construction, painted by Alma or painted by Berthe are not paintable any further. Lets represent these rooms as blocked rooms, and the remaining rooms as free rooms. Thus, without losing any information we can uniquely represent the state of the museum with the following parameters,

- The free rooms.
- Position of Alma.
- Position of Berthe.
- Whose turn is it?
- Present score of the game

We can represent blocked/free rooms of the museum as a binary string, 0 denoting the blocked rooms, and 1 representing the free rooms.

Lets define a function, `CalcScore()`, which takes the binary string, position of Alma and Berthe, and whose turn it is as input parameters and returns the optimal score as output, that is, the minimal possible score if it is Berthe's turn and maximum possible score if it is Alma's turn.

`CalcScore()` recursively calls itself with every reachable state, that is, the player tries to paint all the three neighbouring rooms, thus, recursively call `CalcScore()` for all the valid states.

Lets denote $T(N)$ as time complexity when N rooms are free. In every nested call of `CalcScore()`, the number of free rooms decreases by 1, thus the recursion depth of `CalcScore()` can be at max S^2-2 . A room has at most three neighboring rooms, but if it is not the starting position at least one of them will already be painted because that's where the player came from. Thus, when a player reaches a room, he has only two neighboring rooms to move to. So, every call to `CalcScore()` can generate at max 2 recursive calls to `CalcScore()`.

Thus,

- $T(N) = 2 \times T(N-1)$
- $T(0) = O(1)$

Therefore, the time complexity of the initial configuration is $O(2^{S^2})$. Which although seems to be not good enough for $S = 6$, but in practice, the solution passes well within the time limits as most of the recursion depth will be way less than the presumed maximum. Depends on the implementation details, but the number of calls to `CalcScore()` for the worst case is of the order of 10^5 .

Although not needed for Test Set 2, but, we can use techniques like [Memoization](#) and [Alpha-Beta pruning](#) to further speed up the solution.

Analysis: Yeetzhee

First, note that Pommel can always win the game in a finite number of moves on expectation. To do so for a given input sequence $A_1, A_2, \dots, A_K$, Pommel can simply reroll until the first A_1 dice land on 1, the next A_2 dice land on 2, and so on, with the last A_K dice landing on K . Such an outcome satisfies the group arrangement required by the input; furthermore, on each roll, Pommel has a $1/M$ chance of rolling the needed value for the die she's on. This implies that Pommel is expected to win in $N \times M$ turns. Although this may not be optimal, it demonstrates that Pommel always has a winning strategy.

The basic procedure for calculating the expected value is the same for any approach. Suppose Pommel has already rolled and fixed some (potentially zero) dice. Furthermore, suppose she chooses a set S of dice configurations, such that she will not re-roll her current die if her dice configuration, after her current roll, is in S . If there is a p_i probability that her current dice configuration leads to the i -th element of S , then there is a $1 - \sum p_i$ probability that Pommel will have to re-roll. If we now let e_i equal the expected number of moves to win starting from state i , assuming Pommel plays optimally, we can solve for the expected number of moves to win starting from Pommel's current dice configuration. If we let this quantity be x :

$$x = 1 + (1 - \sum p_i)x + \sum p_i e_i$$

$$x = (1 + \sum p_i e_i) / (\sum p_i)$$

Approaches to the two tests sets differ in how they may enumerate the possible dice configurations and how they find the optimal set S for any given configuration.

Test set 1

For this test set it's sufficient to look at all dice roll results directly. There are at most $\sum_{0 \leq i \leq N} M^i < 60,000$ such possibilities. For a given configuration, performing another roll leads to one of M new dice configurations, so for each configuration we can loop through all $2^M - 1$ possible non-empty subsets of configurations that Pommel could choose to not re-roll on (ignoring subsets that contain configurations that make it impossible for Pommel to reach a winning configuration). For each choice of subset, we compute Pommel's expected turns until winning, and we take the subset with the lowest expected value.

We need to iterate over the dice configurations such that when we're computing the minimum expected value for a configuration C , we know the minimum expected values for each configuration that C can lead to. This can be done, for example, with DFS. The expected value from the empty configuration is our answer. This algorithm runs in $O(M^N \times 2^M \times M)$ time which is sufficient.

As an example, consider the test case $N = 3, M = 2, K = 2, A_1 = 1, A_2 = 2$. Let's compute Pommel's expected turns from winning when she's rolled and locked down a 1. There are two possibilities for the dice configuration after her next turn: $[1, 1]$ and $[1, 2]$, each occurring with probability $1/2$. As a precondition to computing the expected turns to win from $[1]$, we know that, on expectation, it takes two moves to win from $[1, 1]$ and one move to win from $[1, 2]$. We now consider the three possibilities for S : $\{[1, 1]\}$, $\{[1, 2]\}$, and $\{[1, 1], [1, 2]\}$.

$S = \{[1, 1]\}$ leads to the equation $x = 1 + (1 - 0.5)x + 0.5 \cdot 2$, which results in $x = 4$.

$S = \{[1, 2]\}$ leads to the equation $x = 1 + (1 - 0.5)x + 0.5 \cdot 1$, which results in $x = 3$.

$S = \{[1, 1], [1, 2]\}$ leads to the equation $x = 1 + (1 - 0.5 - 0.5)x + 0.5 \cdot 1 + 0.5 \cdot 2$, which results in $x = 2.5$. This is the lowest value, so we now know that if Pommel has rolled a single 1, she will win in 2.5 more turns on expectation if she plays optimally.

Test set 2

For this test set, the number of possible dice roll outcomes, 50^{50} , is far too large to be enumerated in time. We instead focus on the groupings of dice that could possibly lead to the desired configuration. For example, if Pommel rolls $[1, 1, 2, 2]$ or if she rolls $[3, 4, 3, 4]$, either way, she'll have two groups of two dice, which are equivalent for this problem. A dice grouping can be expressed as a $\mathbf{K}$ -tuple of integers $(x_1, x_2, \dots, x_{\mathbf{K}})$ such that the tuple elements are non-decreasing: $x_i \leq x_{i+1}$ for $1 \leq i < \mathbf{K}$. The i -th element of the tuple expresses the number of dice in the i -th smallest dice group, which is zero if there are fewer than $\mathbf{K}$ non-empty groups in the current dice configuration. For example, if $\mathbf{K}=3$ and the dice results so far are $[2, 2, 3, 2, 2, 3]$, the grouping can be expressed as $(0, 2, 4)$.

We need to count how many possible $\mathbf{K}$ -tuples we need to consider to get an estimate of our algorithm's runtime. One simple bound is that we only need to consider a tuple T if the sum of T 's elements is at most $\mathbf{K}$. The total number of tuples we need to consider is therefore at most $\sum_{0 \leq i \leq \mathbf{K}} p(i)$ where $p(i)$ is the number of $\mathbf{K}$ -tuples whose elements are non-decreasing and add to i . $p(i)$ can be calculated with a script, but it is also the i -th [partition number](#). The sum of the first 50 partition numbers is only slightly more than a million, which is very tractable.

A dice roll could lead Pommel from a given configuration to at most $\mathbf{K}$ new configurations. We now need a way of computing the optimal subset of new configurations S to not re-roll that is faster than the naive $O(\mathbf{K} \times 2^{\mathbf{K}})$ approach. Consider the expected value equation shown above. First, note that if for some i , $e_i > x$, removing configuration i from S will decrease the value of x . Similarly, if there is some configuration C with expected value $E[C] < x$ that is not included in S , adding it to S will lower x . This implies that if some configuration C with expected value $E[C]$ is in S , then all states C' with $E[C'] < E[C]$ must also be in S if Pommel is playing optimally. We can therefore sort all configurations reachable from a given configuration so that they're in ascending order of expected turns until winning. It is guaranteed that the optimal S is a prefix of the sorted list, so it can be found by sorting and iterating over the list in $O(\mathbf{K} \log \mathbf{K})$ time.

Combining our improved optimal-subset routine with our bound for the number of $\mathbf{K}$ -tuples (which was in fact quite loose and can be shown to be significantly lower) gives us an algorithm that runs in time for $\mathbf{M}, \mathbf{N} \leq 50$.